~~DAA Assignments~~

DAA - Assignment

Q1) Discuss an efficient method to sort ten "10 digits" numbers in almost constant time?

=> To sort 10 nos of 10 digits each, we use radix sort, which is a non-comparative sorting algorithm.

Method (Radix sort using counting sort):

- Radix sort processes digits from least significant digit (LSD) to most significant digit (MSD).
- For each digit position (0-9), apply counting sort.

Steps:

1) There are 10 digits per number, so perform 10 passes.
2) In each pass:
   sort numbers based on the current digit using counting sort (range 0-9)
3) After the final pass, numbers are fully sorted.

Time Complexity:

- Radix sort: $O(d \times (n+k))$

  $d = 10$ (digits)

  $n = 10$ (numbers)

  $k = 10$ (digit range)

  $O(10 \times (10+10)) = O(200) = O(1)$

Conclusion:

Since d, n, k are constants, the complexity becomes constant time $O(1)$.

Q2) Convert knapsack problem to satisfiability problem using state space tree.

=> We reduce the 0/1 knapsack problem to a boolean satisfiability (SAT) problem.

knapsack definition:

given: items $i = 1$ to n, weight w_i, profit p_i and capacity W.

Step 1: Variables

Define boolean variables: $x_i = 1 \rightarrow$ item included.

$x_i = 0 \rightarrow$ item excluded.

Step 2: Constraints

(a) Weight constraint.

$$\sum w_i x_i \leq W$$

Convert into SAT by encoding into boolean clauses (using binary encoding or CNF transformation)

(b) Profit constraint (optional for decision version)

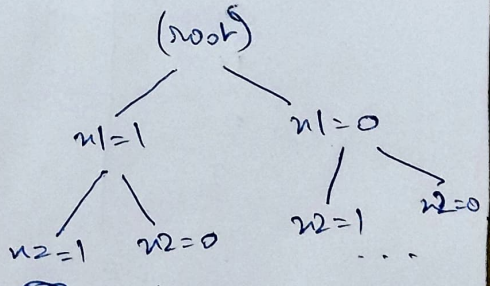
$$\sum p_i x_i \geq P$$

Step 3: State space tree representation

Each level = one item decision (include/exclude)

Leaves = feasible solutions

[Each path gives an assignment of x_i]



Step 4: Conversion to SAT.

- Each valid path corresponds to a satisfying assignment.
- Encode: selection constraints → boolean variables.
  Capacity constraint → CNF clauses.

Conclusion: knapsack is reducible to SAT → proves NP-completeness (decision version).

Q 3) Generate LSP (Longest suffix prefix) / Pi table for the following.

i) aa aaaa bb aa.

index	char	LSP
0	a	0
1	a	1
2	a	2
3	a	3
4	a	4
5	a	5
6	b	0
7	b	0
8	a	1
9	a	2

ii) aaaa

index	char	LSP
0	a	0
1	a	1
2	a	2
3	a	3

iii) abcabcabc

index	char	LSP
0	a	0
1	b	0
2	c	0
3	a	1
4	b	2
5	c	3
6	a	4
7	b	5
8	c	6

Q4) State an example of each (except searching and sorting algorithm)

i) $O(1)$ algorithm

→ Constant time

Ex: Accessing an element in an array.

ii) $O(n^2)$ → quadratic time.

Ex: checking all pairs in an array (nested loop for pair comparison).

iii) $O(n^3)$ → cubic time

Ex: Matrix multiplication (naive approach)

iv) $O(\log n)$ → Logarithmic time

Ex: Finding height of a balanced binary tree (repeated division)

v) $O(n \log n)$ → Linearithmic time

Ex: merging k sorted lists using heap, Building a heap from n elements.

vi) $O(2^n)$ → Exponential time

Ex: Generating all subsets (power set), solving 0/1 knapsack using recursion (without DP)